

คู่มือการทำระบบส่งรูปจาก ESP32-CAM

ไปยัง Google Drive และแจ้งเตือนเข้า LINE (ด้วย Google Apps Script)



ESP32-CAM



Google Apps Script



Google Drive



LINE Messaging API



1 เตรียมฝั่ง LINE (LINE Messaging API)

- 1 สมัคร LINE Developers เข้าสู่ระบบด้วยบัญชี LINE
- 2 สร้าง Provider และ Channel เลือก Messaging API
- 3 ตั้งค่า Channel ตั้งชื่อบอก (เช่น My IoT Bot) ใส่รูปโปรไฟล์และรายละเอียด
- 4 รับ Channel Access Token ไปที่แท็บ Messaging API กด Issue (Long-lived)
- 5 รับ User ID ไปที่ Basic settings คัดลอก Your user ID

สิ่งที่ต้องได้

- Channel Access Token (เก็บไว้ให้ดี)
- Your user ID

2 เตรียมฝั่ง Google (Drive & Apps Script)

2.1 สร้างโฟลเดอร์ใน Google Drive

- 1 สร้างโฟลเดอร์ (เช่น ESP32_Images)
- 2 คัดลอก Folder ID จาก URL

<https://drive.google.com/drive/folders/1aBcDeFg...> → Folder ID 1aBcDeFg...

2.2 สร้าง Google Apps Script (GAS)



2.3 Deploy เป็น Web App

- ✓ ประเภท: เว็บแอป (Web App)
- ✓ การเข้าถึง: ทุกคน (Anyone)
- ✓ กดยอมรับสิทธิ์ (Authorize)
- ✓ คัดลอก Web App URL

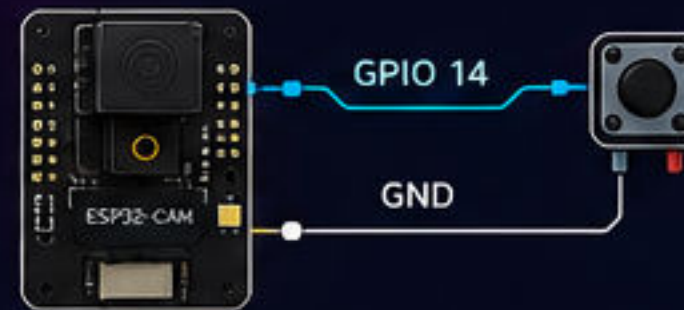
<https://script.google.com/macros/s/.../exec>

สิ่งที่ต้องได้

Web App URL สำหรับ ESP32-CAM

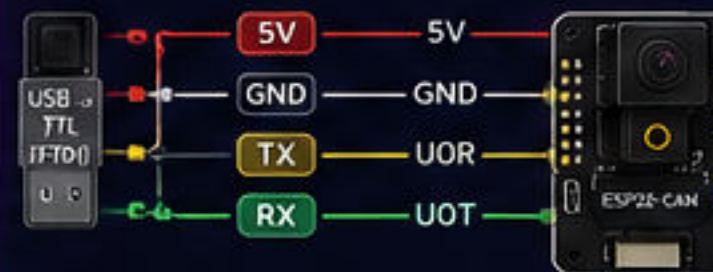
3 เตรียมฝั่งฮาร์ดแวร์ (ESP32-CAM)

3.1 การต่อวงจรปุ่มกด



(ใช้ INPUT_PULLUP ไม่ต้องต่อตัวต้านทานเพิ่ม)

3.2 การต่อบอร์ดเพื่ออัปโหลดโค้ด



ต้องต่อ Jumper ระหว่าง GPIO 0 ↔ GND เพื่อเข้าโหมดอัปโหลด

GPIO 0 — GND

สิ่งที่ต้องใช้

- สวิตช์ปุ่มกด 1 ตัว
- สาย USB to TTL (FTDI)
- Jumper หนึ่งเส้น

4 ฝั่งซอฟต์แวร์ (Arduino IDE)

- ✓ ติดตั้ง Library ที่จำเป็น
 - base64 (Sketch > Include Library > Manage Libraries...)
- ✓ ใส่โค้ดและตั้งค่าตัวแปร

```
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASS";
String GAS_URL = "https://script.google.com/macros/s/.../exec";
```

- ✓ เลือกบอร์ด AI Thinker ESP32-CAM

- ✓ อัปโหลดโค้ด

- 1 กด Upload (ลูกศรขวา)
- 2 ถ้าค้างที่ Connecting... ให้กดปุ่ม RESET ที่บอร์ด 1 ครั้ง
- 3 เสร็จแล้ว 100%
- 4 ถอด Jumper (GPIO 0 ↔ GND) และกด RESET อีก 1 ครั้ง

สิ่งที่ต้องได้

อัปโหลดสำเร็จ บอร์ดทำงานปกติ

5 การทดสอบ (End-to-End Testing)

- 1 เปิด Serial Monitor (115200)
- 2 รอข้อความ
 - ✓ Wi-Fi Connected!
 - ✓ Camera Initialized!
- 3 กดปุ่มที่ต่อกับขา 14
- 4 สังเกตการทำงานใน Serial
 - Button Pressed! Taking a photo...
 - Clearing old buffer (1/2)
 - Clearing old buffer (2/2)
 - Capturing new photo...
 - Encoding image to Base64...
 - Sending to Google Drive & LINE OA...
 - ✓ ส่งภาพสำเร็จ! เข้าไปดูใน LINE ได้เลย
 - Waiting 5 seconds...

5 เช็คลINE Bot ของคุณ จะได้รับรูปภาพทันที!



</> โค้ดสำคัญ (Arduino IDE)

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include "esp_camera.h"
#include <base64.h> // Library base64
```



```
// ... (ตั้งค่า Wi-Fi, Pin, Camera)
void sendImageToGoogle(camera_fb_t * fb) {
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http;
  http.begin(client, GAS_URL);
  http.addHeader("Content-Type", "application/json");
  String imageBase64 = base64::encode(fb->buf, fb->len);
  String payload = "{\"image\":\"\" + imageBase64 + \"\"}";
  int httpCode = http.POST(payload);
  if (httpCode == 200 || httpCode == 302) {
    Serial.println("📷 ส่งภาพสำเร็จ! เข้าไปดูใน LINE ได้เลย");
  } else {
    Serial.printf("❌ Error: %d\n", httpCode);
  }
  http.end();
}
```

ภาพรวมการทำงานของระบบ



จุดเด่นของระบบนี้

- ✓ รูปภาพไม่ค้าง (เคลียร์ Buffer 2 รอบ)
- ✓ ใช้ Google Drive เป็นที่เก็บภาพถาวร
- ✓ ไม่ต้องมีเซิร์ฟเวอร์ของตัวเอง
- ✓ ปลอดภัยด้วย LINE Messaging API
- ✓ ทำงานอัตโนมัติ 24/7



IoT + Cloud + Automation = Smart Notification

คู่มือการทำให้ระบบส่งรูปจาก ESP32-CAM

ไปยัง Google Drive และแจ้งเตือนเข้า LINE (ด้วย Google Apps Script)

เอกสารฉบับนี้จัดทำขึ้นเพื่ออธิบายรายละเอียดขั้นตอนการสร้างระบบรักษาความปลอดภัย (Security System) หรือระบบแจ้งเตือนอัจฉริยะ (Smart Notification) โดยใช้ไมโครคอนโทรลเลอร์ **ESP32-CAM** ร่วมกับบริการคลาวด์จาก **Google (Drive & Apps Script)** และ **LINE Messaging API** โดยมีกระบวนการทำงานหลักคือ เมื่อมีการกดปุ่ม (หรือเซ็นเซอร์ทำงาน) ตัวกล้องจะถ่ายภาพ แปลงไฟล์ภาพเป็นข้อมูล Base64 ส่งไปเก็บที่ Google Drive และส่งข้อความพร้อมรูปภาพแจ้งเตือนเข้าแอปพลิเคชัน LINE ทันที

ภาพรวมการทำงานของระบบ (System Workflow)

- การกระตุ้น (Trigger):** ผู้ใช้กดปุ่ม (หรือเซ็นเซอร์ตรวจจับ) ที่ต่อกับขา GPIO 14 ของบอร์ด ESP32-CAM
- การถ่ายภาพ:** กล้องจะทำการเคลียร์ Buffer เท่า 2 รอบ เพื่อให้ได้ภาพที่เป็นปัจจุบันที่สุด แล้วจึงถ่ายภาพใหม่
- การแปลงข้อมูล:** ภาพที่ถ่ายได้จะถูกแปลงเป็นรหัสตัวอักษร Base64 ภายในบอร์ด ESP32-CAM
- การส่งข้อมูลสู่ Cloud:** บอร์ดจะส่งข้อมูล Base64 ผ่าน Wi-Fi ไปยัง URL ของ **Google Apps Script (Web App)**
- การจัดเก็บลง Drive:** Google Apps Script ทำหน้าที่แปลงข้อมูล Base64 กลับเป็นไฟล์รูปภาพ และบันทึกลงในโฟลเดอร์บน **Google Drive** ที่กำหนดไว้
- การแจ้งเตือน:** Google Apps Script ส่งคำสั่งพร้อมรูปภาพผ่าน **LINE Messaging API** ไปยังห้องแชท หรือ LINE OA
- ผู้รับปลายทาง:** ผู้ใช้งานได้รับการแจ้งเตือนทางแอปพลิเคชัน LINE พร้อมรูปภาพทันที

ส่วนที่ 1: เตรียมฝั่ง LINE (LINE Messaging API)

ขั้นตอนแรกคือการตั้งค่าระบบแชทบอทของ LINE เพื่อใช้เป็นช่องทางในการรับการแจ้งเตือน

- 1. สมัคร LINE Developers:** เข้าสู่ระบบ LINE Developers ด้วยบัญชี LINE ส่วนตัวของคุณ
- 2. สร้าง Provider และ Channel:** สร้าง Provider ใหม่ (เช่น My Smart Home) และสร้าง Channel ประเภท Messaging API
- 3. ตั้งค่า Channel:** กำหนดชื่อบอท (เช่น My IoT Bot) และอัปโหลดรูปโปรไฟล์ตามต้องการ
- 4. รับ Channel Access Token (Long-lived):** ไปที่เว็บ **Messaging API** เลื่อนลงมาด้านล่างสุด แล้วกดปุ่ม Issue เพื่อสร้าง Token สำหรับใช้ยืนยันสิทธิ์ในการส่งข้อความ

- **5. รับ User ID:** ไปที่แท็บ *Basic settings* เลื่อนลงมาด้านล่างสุด เพื่อคัดลอก User ID ของคุณ (ใช้สำหรับระบุผู้รับข้อความ)

💡 **สิ่งที่ต้องได้จากขั้นตอนนี้เพื่อนำไปใช้ต่อ:**

- **Channel Access Token** (เก็บความลับให้ดี ห้ามเผยแพร่)
- **Your user ID**

ส่วนที่ 2: เตรียมฝั่ง Google (Drive & Apps Script)

ระบบนี้ใช้ Google Apps Script เป็นตัวกลาง (Middleware) ทำหน้าที่รับข้อมูลภาพจาก ESP32 เซฟลง Drive และยิง API ไปยัง LINE

2.1 สร้างโฟลเดอร์ใน Google Drive

1. สร้างโฟลเดอร์ใหม่ใน Google Drive ของคุณ (เช่น ตั้งชื่อว่า ESP32_Images)
2. เปิดเข้าไปในโฟลเดอร์นั้น สังเกตที่ URL บนเบราว์เซอร์ จะมีชุดตัวอักษรยาวๆ อยู่หลัง /folders/ นั่นคือ **Folder ID** ให้คัดลอกเก็บไว้

2.2 สร้าง Google Apps Script (GAS)

1. ไปที่ Google Drive กดสร้าง > เพิ่มเติม > **Google Apps Script**
2. ระบบจะเปิดหน้าต่างเขียนโค้ดขึ้นมา โค้ดในส่วนนี้จะทำหน้าที่รับข้อมูล JSON จาก ESP32, แปลง Base64 เป็นไฟล์ภาพ, บันทึกลง Folder ID ที่เราเตรียมไว้ และส่ง Request ไปที่ LINE API โดยใช้ Channel Access Token และ User ID ที่เตรียมไว้ในส่วนที่ 1

2.3 Deploy เป็น Web App

1. เมื่อเขียนโค้ดเสร็จ ให้กดที่ปุ่ม **การทำให้ใช้งานได้ (Deploy) > การทำให้ใช้งานได้รายการใหม่ (New deployment)**
2. **ประเภท (Type):** เลือก เว็บแอป (Web App)
3. **การเข้าถึง (Who has access):** สำคัญมาก! ต้องเลือกเป็น **ทุกคน (Anyone)** เพื่อให้ ESP32-CAM สามารถส่งข้อมูลเข้ามาได้โดยไม่ต้องล็อกอิน
4. กดยอมรับสิทธิ์ (Authorize access) ให้เรียบร้อย

💡 สิ่งที่ต้องได้จากขั้นตอนนี้เพื่อนำไปใช้ต่อ:

- **Web App URL** (เริ่มต้นด้วย `https://script.google.com/macros/s/.../exec`) สำหรับนำไปใส่ในโค้ด Arduino

ส่วนที่ 3: เตรียมฝังฮาร์ดแวร์ (ESP32-CAM)

การต่อวงจรสำหรับโปรเจกต์นี้แบ่งเป็น 2 ส่วนหลัก คือวงจรสำหรับการอัปโหลดโค้ด และวงจรสำหรับการใช้งานจริง (ต่อสวิตช์)

3.1 การต่อวงจรเพื่ออัปโหลดโค้ด

เนื่องจากบอร์ด ESP32-CAM ส่วนใหญ่ไม่มีพอร์ต USB ติดตั้งมาในตัว จึงต้องใช้โมดูล **USB to TTL (FTDI)** ในการอัปโหลดโค้ดเข้าบอร์ด

ขาโมดูล USB to TTL (FTDI)	ขา ESP32-CAM	หมายเหตุ
5V	5V	แหล่งจ่ายไฟ
GND	GND	กราวด์ระบบ
TX	U0R	ส่งข้อมูลจากคอมเข้าบอร์ด
RX	U0T	รับข้อมูลจากบอร์ดเข้าคอม

ข้อควรระวังสำคัญ (Flash Mode):

ในการอัปโหลดโค้ด คุณ **ต้องต่อ Jumper ระหว่างขา GPIO 0 กับขา GND** เพื่อบังคับให้บอร์ดเข้าสู่โหมดอัปโหลด (Flash Mode) หากไม่ต่อขั้นตอนนี้ จะอัปโหลดโค้ดไม่ผ่าน

3.2 การต่อวงจรปุ่มกด (Trigger)

ใช้สวิตช์ปุ่มกด 1 ตัว ต่อเข้ากับบอร์ดดังนี้:

- **ขาหนึ่งของสวิตช์:** ต่อเข้ากับขา **GPIO 14** ของ ESP32-CAM
- **อีกขาหนึ่งของสวิตช์:** ต่อเข้ากับขา **GND**

หมายเหตุ: ในโค้ด Arduino จะตั้งค่าขา 14 เป็นโหมด INPUT_PULLUP จึงไม่จำเป็นต้องต่อตัวต้านทาน (Resistor) เพิ่มเติมภายนอก

ส่วนที่ 4: ฟังซอฟต์แวร์ (Arduino IDE)

การเขียนโปรแกรมควบคุม ESP32-CAM ให้ทำงานร่วมกับระบบที่เตรียมไว้

1. **ติดตั้ง Library ที่จำเป็น:** ไปที่เมนู *Sketch > Include Library > Manage Libraries...* ค้นหาและติดตั้งไลบรารี base64 (ใช้สำหรับเข้ารหัสรูปภาพก่อนส่ง)
2. **ตั้งค่าตัวแปรในโค้ด:** นำข้อมูลเครือข่ายและ URL ที่ได้เตรียมไว้มาแก้ไขในโค้ดให้ถูกต้อง
 - `const char* ssid = "YOUR_WIFI_NAME";` (ชื่อ Wi-Fi 2.4GHz)
 - `const char* password = "YOUR_WIFI_PASSWORD";` (รหัสผ่าน Wi-Fi)
 - `String GAS_URL = "https://script.google.com/macros/s/.../exec";` (Web App URL ที่ได้จากส่วนที่ 2.3)
3. **เลือกบอร์ด:** ไปที่เมนู *Tools > Board* เลือก AI Thinker ESP32-CAM (หากไม่มีให้ติดตั้ง ESP32 Board Manager ก่อน)
4. **อัปโหลดโค้ด:** กดปุ่ม *Upload* (ลูกศรขวา) หากค้างที่คำว่า *Connecting...* ให้กดปุ่ม **RESET** ที่ตัวบอร์ด ESP32-CAM 1 ครั้ง
5. **เมื่ออัปโหลดเสร็จแล้ว 100%:** ให้ ถอดสาย Jumper (GPIO 0 ↔ GND) ออก แล้วกดปุ่ม **RESET** ที่บอร์ดอีก 1 ครั้งเพื่อให้โค้ดเริ่มทำงาน

โครงสร้างโค้ดสำคัญ (Arduino IDE)

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include "esp_camera.h"
#include <base64.h> // Library สำหรับเข้ารหัสภาพ

// ฟังก์ชันสำหรับถ่ายและส่งรูปภาพ
void sendImageToGoogle(camera_fb_t * fb) {
    WiFiClientSecure client;
    client.setInsecure(); // ละเว้นการตรวจสอบ SSL Certificate
    HTTPClient http;
    http.begin(client, GAS_URL);
    http.addHeader("Content-Type", "application/json");

    // เข้ารหัสรูปภาพ (Buffer) เป็น Base64
    String imageBase64 = base64::encode(fb->buf, fb->len);

    // สร้าง Payload รูปแบบ JSON
    String payload = "{\"image\":\"" + imageBase64 + "\"}";

    // ส่งข้อมูลแบบ POST
    int httpCode = http.POST(payload);
    if (httpCode == 200 || httpCode == 302) {
        Serial.println("✅ ส่งภาพสำเร็จ! เข้าไปใน LINE ได้เลย");
    } else {
        Serial.printf("❌ Error: %d\n", httpCode);
    }
    http.end();
}
```

ส่วนที่ 5: การทดสอบ (End-to-End Testing)

เมื่อทุกอย่างพร้อม ให้ทำตามขั้นตอนต่อไปนี้เพื่อทดสอบระบบอย่างเต็มรูปแบบ:

- เปิดหน้าต่าง **Serial Monitor** ใน Arduino IDE (ตั้งค่า Baud Rate ที่ 115200)
- รอนกระทั่งปรากฏข้อความยืนยันการเชื่อมต่อ:
 - ✅ Wi-Fi Connected!
 - ✅ Camera Initialized!

3. ทำการทดสอบโดย กดปุ่มที่ต่อกับขา 14

4. สังเกตการทำงานใน Serial Monitor ซึ่งระบบจะแสดงขั้นตอนต่างๆ ได้แก่:

- Button Pressed! Taking a photo... (ตรวจพบการกดปุ่ม)
- Clearing old buffer (1/2) (เคลียร์ภาพเก่าในหน่วยความจำรอบแรก)
- Clearing old buffer (2/2) (เคลียร์ภาพเก่าในหน่วยความจำรอบสอง)
- Capturing new photo... (กำลังบันทึกภาพปัจจุบัน)
- Encoding image to Base64... (แปลงไฟล์ภาพเป็นข้อความ)
- Sending to Google Drive & LINE OA... (กำลังส่งข้อมูลไปที่ Web App)
-  ส่งภาพสำเร็จ! เข้าไปดูใน LINE ได้เลย (กระบวนการสมบูรณ์)
- Waiting 5 seconds... (หน่วงเวลาสำหรับการกดซ้ำ)

5. เช็ก **LINE Bot ของคุณ**: เปิดแอปพลิเคชัน LINE คุณจะได้รับความแจ้งเตือนพร้อมรูปถ่ายที่ถ่ายจาก ESP32-CAM กันที!

จุดเด่นของระบบนี้

- **รูปภาพไม่ค้าง (Real-time Accuracy)**: มีการเขียนโค้ดเพื่อเคลียร์ Buffer รูปภาพในตัวกล้อง 2 รอบก่อนการถ่ายจริงเสมอ ทำให้มั่นใจได้ว่าภาพที่ส่งมาคือเหตุการณ์ปัจจุบัน ณ วินาทีนั้น ไม่ใช่ภาพเก่าที่ค้างในหน่วยความจำ
- **ใช้ Google Drive เป็น Storage**: ถือเป็นพื้นที่เก็บภาพถาวร (Archive) โดยไม่ต้องเสียเงินเช่าเซิร์ฟเวอร์ สะดวกต่อการเรียกดูย้อนหลัง
- **Serverless Architecture**: ไม่ต้องตั้งเซิร์ฟเวอร์ของตัวเอง (No backend server required) ทำงานผ่าน Google Cloud อินฟราสตรัคเจอร์โดยตรง
- **ปลอดภัยและแจ้งเตือนเร็ว**: ทำงานอัตโนมัติตลอด 24 ชั่วโมง และข้อมูลถูกส่งผ่าน API มาตรฐานอย่างปลอดภัย